43.PPO

21 March 2026 16:37

* Notes was written when my RL knowledge wasn't that great, Apologies in advance for high verbosity

Proximal policy optimization Algorithms

→ new family of policy gradient methods for reinforcement learning
↓
which alternate bw sampling data through interaction with the environment and optimizing a "surrogate" objective function using stochastic gradient ascent

Claims
Simpler than trust-region methods
more sample efficient
empirically better

standard policy gradient methods perform one gradient update per data sample
→ A novel objective function that enables multiple epochs of minibatch updates

improve policy gradients methods to use data more efficiently while staying stable

Standard policy gradient methods uses data once then throws it away

New objective
→ allows multiple updates on same data
→ without breaking training

Data depends on the policy that generated it!

At time t
π_old → generates trajectories → data D
Then you update
π_new ← optimize using D

} if we do many updates on the same data
π_new moves far away from π_old
but the data comes from π_old

Mismatch
we're doing
$E_{\sim}[\text{data} \sim \pi_{old}][\text{using } \pi_new]$
We actually want
$E_{\sim}[\text{data} \sim \pi_{new}][\quad]$

assume
Data → samples from one distⁿ (π_old)
You optimize a different distⁿ (π_new)

if π_new moves too far gradients become wrong/biased/unstable

→ if we ignore this → training becomes unstable
→ we need small, controlled updates to the policy

Gradient lens →
gradient updated using π_old parameters π_new } mismatch
data no longer representative
Gradients point in the wrong direction

The data distribution depends on the model itself

→ need for scalable, robust, efficient RL Methods

Vanilla policy gradients → poor data efficiency

TRPO → complex
DQN → works well in discrete action spaces not in continuous control benchmarks

An objective that does not overestimate improvements

PPO avoids second order methods (TRPO) → expensive, complex, hard to scale
↳ no Hessian
↳ no expensive curvature computation
↳ just standard SGD/adam

ppo aims to match TRPO's stability while using only first-order optimization making it simpler & scalable

$$\hat{g} = E_t\left[\nabla_\theta \log \pi_\theta(a_t|s_t)\hat{A}_t\right]$$

how probability changes w.r.t parameters
gradient signal direction

log probability of action under policy
Policy = distribution over actions

Advantage = how good an action is relative to baseline

$A_t > 0$ → action better than expected → increase probability
$A_t < 0$ → worse → decrease probability

They construct an objective whose gradient matches the estimator

$$L^{PG} = E_t\left[\log \pi_\theta(a_t|s_t)\hat{A}\right]$$

gradient flow → policy → log policy → wt by advantage → gradient

information flow environment → rewards → advantage → policy

In TRPO
maximize surrogate objective
constrain KL $K \le \delta$

$$\text{maximize } E_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{old}(a_t|s_t)}\hat{A}_t\right]$$

Subject to –
$$\hat{E}_t\left[KL\left[\pi_{\theta_{old}}(\cdot|s_t)\ \pi_\theta(\cdot|s_t)\right]\right] \le \delta$$

Multiple updates → policy drift → instability

TRPO → constrain KL divergence (but second order)

Why KL divergence not θ difference

because small param change → big distⁿ change

PPO good –
get same stability without complexity

KL measures diff bw π_{old} & π_{new} behaviors

Objective $\max E\left[\frac{\pi_\theta}{\pi_{old}} A_t\right]$

Constraint $KL(\pi_{old} \| \pi_{new}) \le \delta$

π_{new}/π_{old} tells how much policy changed

Why KL divergence not θ difference

because small param chang → big distⁿ change
large param chang → small distⁿ chang

parameter space ≠ behavior space

TRPO constrains behavior change Not parameter change

KL measures diff b/w π_{old} & π_{new}

behavior
action probabilities
data distribution

Policy lives in probability distribution space

π_{new}/π_{old} tells how much policy changed

KL constraint keeps new policy close to the old

× Step size is defined in distribution space not parameter space

They replace constraint with penalty

Hard constraint (TRPO) $K \le \delta$

Soft penalty Objective $- \beta * KL$
↳ choosing this is hard

Conjugate gradient
↳ used to solve constrained optimization efficiently

3 Clipped surrogate objective

Conservative policy iteration

let $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ $\quad r(\theta_{old}) = 1$ $\quad L^{CPI}(\theta) = E_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t\right] = E_t\left[r_t(\theta)\hat{A}_t\right]$

without constraint, maximization of L^{CPI} would lead to excessively large policy update

penalize changes to the policy that move $\hat{r}_t(\theta)$ away from 1

$$L^{CLIP}(\theta) = E\left[\min\left(\hat{r}_t(\theta)\hat{A}_t,\ \underbrace{\text{clip}\left(r_t(\theta), 1-\varepsilon, 1+\varepsilon\right)\hat{A}_t}\right)\right]$$

a lower bound (pessimistic bound)

ε = hyperparameter

first term is L^{CPI}

modifies surrogate objective by clipping the probability ratio which removes the incentive of moving r_t outside of the interval $[1-\varepsilon, 1+\varepsilon]$

4 Adaptive KL penalty coefficient

Another approach is to use a penalty on KL divergence and to adapt the penalty coefficient so that we achieve some target value of the KL divergence during each policy update

performed worse in experiments but an important baseline

* Using several epochs of minibatch SGD, perform following steps in each policy update

$$L^{KLPEN}(\theta) = E_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t - \beta KL\left[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)\right]\right]$$

Compute $d = E_t\left[KL\left[\pi_{\theta_{old}}, \pi_\theta\right]\right]$ $\quad$ updated β is used for next policy update

if $d < d_{target}/1.5$, $\beta \leftarrow \beta/2$

if $d > d_{target} \times 1.5$, $\beta \leftarrow \beta \times 2$

→ we want to maximize $r_t(\theta)A_t$

But maximizing this aggressively pushes r away from 1

Case 1 $A_t > 0$
maximize $r * A$ → push $r \to \infty$
increase probability a lot

Case 2 $A_t < 0$
maximize $r * A$ → push $r \to 0$
decrease probability a lot

large policy updates instability

$L^{CLIP} = E\left[\min\left(rA, \text{clip}(r, 1-\epsilon, 1+\epsilon)A\right)\right]$

Clipping removes incentives for large updates

$A_t > 0$ increase r
without clipping
$r\uparrow \Rightarrow$ obj ↑
if $r \le 1+\epsilon$ normal
if $r > 1+\epsilon$ clipped
Compare
rA vs $(1+\epsilon)A$
Since $r > 1+\epsilon$
$rA > (1+\epsilon)A$
even if increasing r would improve objective, we ignore that improvement

$A_t < 0$ decrease r
r↓ objective ↑ (less negative)
with clipping
if $r < (1-\varepsilon)$ clipped
compare rA vs $(1-\varepsilon)A$
Since $A < 0$ signs flip
$r < (1-\varepsilon) \Rightarrow rA > (1-\varepsilon)A$
min picks $(1-\varepsilon)A$ $\quad$ even if decreasing r helps objective we STOP at beyond a limit

Optimization is pessimistic
it will only trust improvements that don't require large policy changes

Why do we need $\min(rA, \text{clip}(r)A)$ $\quad$ PPO replaces a hard KL constraint with a clipped objective that implicitly limits policy updates

The optimizer will exploit whatever objective we give it

The optimizer will exploit whatever objective we give it

if only $L = clip(r)A$

(1) $A > 0$
$clip(r) \le (1+\varepsilon)$
Objective becomes $(1+\varepsilon)A$
Constant
even if $r >> (1+\varepsilon)$
The objective doesn't decrease

Consequence
the optimizer has no penalty for pushing r far beyond the clip

it can still move the policy aggressively because objective doesn't punish it
just saturates

clipping alone removes incentive but does NOT discourage bad updates

$\min(r, 1-\varepsilon, 1+\varepsilon)A$
picks worse (lower) objective

Case $A>0$, $r>1+$
unclipped $rA \to$ large
clipped $(1+\varepsilon)A$
min = clipped

if optimizer tries push r further
$r\uparrow$ $rA\uparrow$ $\to$ but min stays const

with min() the optimizer won't benefit from going outside the safe region

When r goes outside $[1+\varepsilon, 1-\varepsilon]$

Objective becomes constant
→ no gradient
→ no incentive to move further

← Inside trust region
normal optimization
Outside → flat plateau

Beyond clip → gradient = 0

a neural network architecture that shares parameters between the policy surrogate and a value function error term

A loss function that combines the policy surrogate & a value function error term

Adding an entropy bonus to ensure sufficient exploration

$$L_t^{CLIP+VF+S}(\theta) = \hat{E}_t\left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\right]$$

entropy bonus

constants

Squared error loss
$(V_\theta(s_t) - V_t^{target})^2$

What is advantage?

$A_t = Q(s_t, a_t) - V(s_t)$

How much better was this action than expected?

why needed?
without it
reward → very noisy → unstable gradients

in PPO A_t → signal that tells increase or decrease probability

Advantage estimator used by [Mni+16] $\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \quad + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T)$

$t \to$ time index in $[0, T]$

$T \to$ trajectory segment

A truncated version $\lambda = 1$

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \quad + (\gamma\lambda)^{T-t+1}\delta_{T-1}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Value function Loss –

Squared loss between predicted and target

$L^{VF} = (V(s_t) - V^{target})^2$

to compute advantage
→ Value function is a baseline estimator
→ reduces variance of policy gradient

Algorithm for fixed length trajectory segments

each iteration each of (N) parallel actors collect T timesteps of data

We construct the surrogate loss on these NT timesteps & optimize with SGD for K epochs

$S[\pi(s_t)]$

Encourages high entropy distribution

Effect prevents policy collapse → keeps randomness

pushes probabilities to be less peaked

$L = L^{CLIP} - L^{VF} c_1 + c_2 S$

$S \to$ exploration

$L^{VF} \to$ Value Learning

$L^{CLIP} \to$ policy improvement

Algo PPO → Actor Critic style

for iteration = 1, 2 do
for actor = 1, 2 . N do
Run policy $\pi_{\theta_{old}}$ in env for T time steps
Compute advantage estimates $\hat{A}_1 \ldots \hat{A}_T$
end for
Optimize surrogate L wrt θ, with K epochs & minibatch size $M \le NT$
$\theta_{old} \leftarrow \theta$
end for

multiple epochs same data

SGD ∇ gradient Ascent here

They are maximizing objective

We minimize loss in DL frameworks

Advantage reduces variance, gives direction

Value function needed to compute advantage

Entropy prevents collapse

We minimize loss in dL frameworks

they convert maximize L = minimize -L

paper says gradient ascent

implementation uses gradient descent on negative loss

Value function needed to compute advantage

entropy prevents collapse

Clipping stabilizes updates

Multiple epochs improve sample efficiency

Actor-critic → decides actions (policy π)
→ evaluates states (value V)

two coupled functions

→ Why two components

problem in policy gradient

reward → direct policy update

Actor outputs probability of actions $\pi_\theta(a|s)$

this is what we optimize

Critic (value function) $V_\phi(s)$

predicts expected future rewards from state s

Very noisy

high variance

unstable learning

critic → instead of raw reward

use relative signal

$A_t = Q(s,a) - V(s)$

Actor update

$\nabla \log \pi(a|s) \times A_t$

critic update

$(V(s) - \text{target})^2$

Value function

$V(s) = E[\text{future rewards from } s]$

baseline expectation of how good a state is

A_t = reward − expectation

without entropy

policy becomes very confident

↳ picks same action always → premature convergence

↳ locks into suboptimal policy

Failure mode

early convergence to bad policy

no exploration → No exploration

Never discovers better actions

$S[\pi(s)] = -\sum_a \pi(a|s) \log \pi(a|s)$

→ when probabilities → 0 or 1

log π gradients → vanish

learning slows / stops

full loop (PPO Content)

1. Actor generates data
   π_{old} → trajectories
2. Critic evaluates
   $V(s)$ → baseline
3. Compute advantage
   A_t = reward signal − $V(s)$
4. Update actor
   maximize PPO clipped objective
5. Update critic
   minimize value prediction error
6. Add entropy
   encourage exploration

Actor → policy learner

Critic → signal denoiser

Entropy → exploration regulator

Raw rewards mix → action quality & state difficulty

Reward → absolute signal

Advantage → relative signal

Same action, different states

hard state → low reward

easy state → high reward

reward alone can't tell

was the action good?